

A Survey of Deep Learning-Based Software Defect Prediction Techniques

Mogage Nicolae

November 27, 2025

Abstract

Software Defect Prediction (SDP) has evolved from a niche academic interest into a critical component of modern Software Quality Assurance (SQA). As software systems grow in scale and complexity, traditional manual verification and static analysis methods have become insufficient. This survey investigates the paradigm shift from manual metric-based machine learning to automated feature extraction via Deep Learning (DL). We critically examine the efficacy of diverse neural architectures—including Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, Graph Neural Networks (GNNs), and Transformer-based Large Language Models (LLMs)—in capturing the semantic and structural nuances of source code. The survey analyzes the evolution of code representation, from simple token embeddings to complex Code Property Graphs (CPGs), and evaluates their impact on prediction granularity. Furthermore, we address persistent challenges such as class imbalance, data leakage, and the computational costs of deploying large models in Continuous Integration (CI) pipelines. The study concludes by forecasting the integration of agentic AI for autonomous defect verification and repair.

1 Introduction

The frequency of software in critical infrastructure - spanning healthcare, finance, aviation, and telecommunications - imposes a stringent requirement for reliability. However, the software development process is inherently complex and human-centric, inevitably leading to defects that can result in catastrophic economic and operational consequences. Recent global outages caused by faulty updates highlight the precarious nature of digital infrastructure and the necessity for proactive defect detection [1]. Software Defect Prediction (SDP) addresses this challenge by leveraging historical data to predict the likelihood of future faults, enabling organizations to prioritize rigorous testing for high-risk modules before deployment [4].

Historically, SDP relied on statistical correlations with static code metrics (e.g., Lines of Code, Halstead complexity). While foundational, these methods operated on the assumption that structural complexity is a direct proxy for defectiveness. This simplification often failed to capture the subtle semantic logic errors - such as incorrect variable usage or data flow anomalies - that characterize modern software failures [5]. The appearance of Deep Learning (DL) has revolutionized this field by enabling the automatic extraction of complex, latent features directly from raw source code, bridging the “semantic gap” that limited previous generations. While prior surveys such as [5] provided valuable early taxonomies of these techniques, the rapid emergence of LLMs and line-level prediction methods since 2021 necessitates an updated synthesis.

1.1 Research Objectives

This paper synthesizes the state-of-the-art in DL-based SDP by addressing the following Research Questions (RQs):

- **RQ1:** How have code representation techniques evolved to support the specific requirements of DL architectures?
- **RQ2:** Which DL architectures yield the highest performance for specific tasks, and what are the trade-offs regarding computational cost and latency?
- **RQ3:** What are the critical methodological challenges (e.g., data leakage, class imbalance) and future directions in the era of Generative AI?

2 The Evolution of Defect Prediction Paradigms

SDP is categorized under “Methods for Analysis and Evaluation” in software engineering research. To appreciate the current capabilities of DL, one must understand the limitations of the paradigms that preceded it.

2.1 The Era of Static Metrics and Machine Learning

The first generation of SDP (1970s–2000s) relied on **static metrics**. Researchers employed linear regression to model relationships between metrics like McCabe’s Cyclomatic Complexity and defect density. While these metrics provided a quantitative baseline, they were criticized for treating code as a structural skeleton devoid of meaning.

The second generation (2000s–2015) introduced **Machine Learning (ML)** classifiers such as Support Vector Machines (SVM), Random Forest, and Naïve Bayes. These models used static metrics as feature vectors to perform binary classification (buggy vs. clean). While effective for coarse-grained prediction, they required extensive manual feature engineering and struggled to generalize across different projects due to the heterogeneity of metric distributions.

2.2 The Deep Learning Revolution

The current generation (2015–Present) is defined by **Representation Learning**. DL models do not require manual feature definition. Instead, they accept raw representations of code—tokens, syntax trees, or graphs—and automatically learn the features that are most discriminative for defect prediction [5]. This shift aligns SDP with broader AI trends in Natural Language Processing (NLP) and Computer Vision. Just as CNNs learn to recognize edges in images, DL models in SDP learn to recognize “code smells” and dangerous data flow patterns directly from the source code.

3 Deep Learning Architectures and Code Representation

The effectiveness of any DL model is inextricably linked to how source code is represented. Unlike natural language, code has a dual nature: it is both a sequence of tokens and a hierarchical structure governed by strict grammar.

3.1 Sequential and Tree-Based Models

Treating code as a sequence allows researchers to leverage standard NLP architectures.

- **CNNs for Local Features:** Convolutional Neural Networks (CNNs) treat code as a matrix of token embeddings. Filters slide over this matrix to detect local features, such as specific combinations of operators or keywords. While efficient, standard CNNs struggle to capture long-range dependencies between variables defined far apart in a file.
- **LSTMs for Long-Range Context:** Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) networks, utilize gating mechanisms to maintain a “cell state,” allowing them to remember information over long sequences. This is crucial for linking a variable’s declaration to its potentially erroneous usage hundreds of lines later [1].
- **AST-Based Tree-LSTMs:** To respect the grammatical structure of code, researchers utilize Abstract Syntax Trees (ASTs). Models like Tree-LSTMs perform computations over the tree structure rather than a linear sequence, effectively filtering out non-functional elements like whitespace.

3.2 Graph Neural Networks (GNNs)

Graph-based approaches represent the frontier of structural defect prediction. Code is transformed into a **Code Property Graph (CPG)**, a joint representation that fuses the AST, Control Flow Graph (CFG), and Program Dependence Graph (PDG) [6].

- **Mechanism:** GNNs employ a message-passing protocol. Each node in the graph (representing a statement or variable) aggregates information from its neighbors. Through multiple layers, the network learns a representation that reflects the flow of data and control.
- **Impact:** This approach allows for the detection of complex vulnerabilities, such as null pointer dereferences, which are defined by reachability in the PDG rather than proximity in the text file. Empirical studies show GNNs achieving significant accuracy gains over token-based baselines.

3.3 Transformers and Large Language Models (LLMs)

The introduction of the Transformer architecture has precipitated a semantic revolution.

- **Pre-Trained Models (CodeBERT):** Models like CodeBERT are pre-trained on massive bimodal corpora (code and natural language). They excel at semantic tasks and have shown exceptional performance in Cross-Project Defect Prediction (CPDP) by learning universal defect patterns [3].
- **Generative LLMs:** Recent Large Language Models (e.g., GPT-4, LLaMA) introduce “in-context learning.” Unlike binary classifiers, these models can explain their predictions and generate patches. However, they face challenges with “context rot” and high inference costs compared to specialized lightweight models [8].

4 Benchmarking and Comparative Analysis

Evaluating SDP models requires robust metrics. The field has largely abandoned Accuracy due to class imbalance, favoring **F1-Score** and **AUC** (Area Under the ROC Curve) [10].

4.1 Performance vs. Cost Trade-offs

Table 1 presents a comparative overview. While LLMs offer versatility, specialized models often provide a better cost-performance ratio for high-volume binary prediction tasks.

Table 1: Comparative overview of SDP techniques. Metrics are illustrative averages derived from surveyed literature.

Paradigm	Key Architectures	Input Representation	Strengths	Weaknesses	Typical F1 / AUC
Traditional ML	Random Forest, SVM	Static Metrics	Fast, interpretable	Misses semantics	AUC $\sim$ 0.70 - 0.75
Sequential DL	Bi-LSTM, CNN	Token Sequences	Captures context	Vanishing gradient	Accuracy $\sim$ 87-90% [1]
Graph-Based	GNN, GCN	CPG (AST+CFG+PDG)	Robust data flow	High compute cost	F1 $\sim$ 0.77 [6]
Just-In-Time	DeepJIT, CC2Vec	Commit Diff + Message	Actionable, fast	Data dependent	AUC $\sim$ 0.75 - 0.80 [2]
Transformer	CodeBERT	Tokens + NL	SOTA semantics	High resource usage	F1 $>$ 0.80 [3]
Generative LLM	GPT-4, LLaMA	Raw Code Context	Few-shot, repair	High latency	F1 $\sim$ 0.84 [8]

4.2 Specialized Paradigms

- **Just-In-Time (JIT):** JIT prediction operates at the commit level. Approaches like **DeepJIT** and **CC2Vec** analyze code changes (diffs) and commit messages to flag defects at the moment of introduction [7].
- **Line-Level Localization:** The ultimate goal is not just to flag a file, but to pinpoint the line. Models like **DeepLineDP** use attention mechanisms to assign risk scores to individual tokens, allowing developers to focus their code review on specific lines [4].

5 Critical Challenges and Future Directions

Despite technical strides, several methodological and practical challenges threaten the validity and deployment of DL-based SDP models.

5.1 Methodological Pitfalls

- **Class Imbalance:** Software datasets are inherently imbalanced (defects are rare). Standard training leads to biased models. Techniques like **GAN-based Oversampling** have emerged as superior to traditional SMOTE, generating diverse, realistic synthetic defect samples [9].
- **Data Leakage:** Improper splitting of time-series data is a pervasive issue. In JIT prediction, training on future commits and testing on past commits violates causality. Strict chronological splitting is mandatory.
- **Interpretability (XAI):** Developers distrust “black box” predictions. Explainable AI methods like **LIME** and **SHAP** are essential for adoption, providing a visual rationale for the alert [?].

5.2 Future Work: Agentic AI

The future of SDP lies in **Agentic AI**. We predict a shift from passive prediction tools to autonomous agents that function as virtual QA engineers. These agents will not only flag defects but actively verify them by generating unit tests, engage in dialogue with developers to clarify ambiguous code, and autonomously propose patches.

6 Conclusion

The trajectory of Software Defect Prediction has irrevocably moved toward the deep, semantic understanding of code enabled by neural networks. This survey demonstrates that DL architectures—specifically LSTMs for sequential logic, GNNs for structural data flow, and Transformers

for semantic context—consistently outperform traditional methods. While challenges regarding data quality and model transparency remain, the integration of these models into agentic workflows offers a path toward self-healing software systems.

References

- [1] Albattah, W., & Alzahrani, M. Software Defect Prediction Based on Machine Learning and Deep Learning Techniques: An Empirical Approach. *AI*, 5(4), 1743–1758, 2024.
- [2] Hoang, T., Dam, H. K., Kamei, Y., Lo, D., & Ubayashi, N. DeepJIT: An End-to-End Deep Learning Framework for Just-In-Time Defect Prediction. *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, 34–45, 2019.
- [3] Feng, Z., Guo, D., Tang, D., et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1536–1547, 2020.
- [4] Pornprasit, C., & Tantithamthavorn, C. K. DeepLineDP: Towards a Deep Learning Approach for Line-Level Defect Prediction. *IEEE Transactions on Software Engineering*, 49(1), 84–98, 2023.
- [5] Akimova, E. N., Bersenev, A. Y., et al. A Survey on Software Defect Prediction Using Deep Learning. *Mathematics*, 9(11), 1180, 2021.
- [6] Althiban, A. S., Alharbi, H. M., et al. Graph Neural Network for Source Code Defect Prediction. *IEEE Access*, 10, 10402–10415, 2022.
- [7] Pornprasit, C., Tantithamthavorn, C. K., & Jirapanthong, W. JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction. *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*, 2021.
- [8] Wang, J., Huang, Z., et al. Fine-Tuned Large Language Models in Software Defect Prediction. *ResearchGate*, 2025.
- [9] Alsaeedi, A., & Khan, M. Z. Leveraging Ensemble Learning with Generative Adversarial Networks for Imbalanced Software Defects Prediction. *Applied Sciences*, 13(24), 13319, 2023.
- [10] Shepperd, M., Bowes, D., & Hall, T. Researcher Bias: The Use of Criteria for Collating Software Defect Prediction Results. *arXiv preprint arXiv:2003.01182*, 2020.